

LAPORAN TUGAS BESAR
INTEGRASI APLIKASI ENTERPRISE



Sistem Informasi
Telkom University
Surabaya

Disusun oleh:

1. Salwa Asshyfa Airuddin (102062400008)
2. Vincent Nathanael Ivanto (102062400014)
3. Afandi Himawan (102062400025)
3. Niken Kazia Situmeang (102062400116)

PROGRAM STUDI S1 SISTEM INFORMASI

FAKULTAS REKAYASA INDUSTRI

TELKOM UNIVERSITY SURABAYA

2026

DAFTAR ISI

A

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi mendorong kebutuhan akan sistem yang mampu menangani proses bisnis secara lebih fleksibel, mudah dikembangkan, dan memiliki skalabilitas yang baik. Arsitektur monolitik yang menggabungkan seluruh fungsi aplikasi ke dalam satu layanan sering kali menimbulkan kendala dalam pemeliharaan, pengembangan, serta integrasi ketika sistem semakin kompleks. Oleh karena itu, arsitektur microservice menjadi salah satu pendekatan yang banyak digunakan dalam pengembangan aplikasi modern.

Arsitektur microservice membagi sistem menjadi beberapa layanan independen yang saling berkomunikasi melalui mekanisme integrasi tertentu. Setiap layanan memiliki tanggung jawab yang spesifik sehingga pengembangan, pemeliharaan, dan deployment dapat dilakukan secara lebih mudah. Selain itu, pendekatan ini memungkinkan setiap layanan menggunakan basis data yang terpisah dan dapat dikembangkan secara mandiri tanpa mempengaruhi layanan lainnya.

Pada proyek ini dikembangkan sebuah Warkop Management System yang menerapkan konsep Enterprise Application Integration (EAI) berbasis microservice. Sistem terdiri atas beberapa layanan independen, yaitu User Service, Product Service, Order Service, dan Payment Service. Integrasi antar layanan dilakukan menggunakan komunikasi sinkron melalui REST API, komunikasi asinkron berbasis message broker RabbitMQ, serta penyediaan data yang fleksibel menggunakan GraphQL melalui Hasura. Seluruh layanan dijalankan menggunakan container Docker sehingga proses deployment dan pengelolaan sistem menjadi lebih mudah dan terstruktur.

Penerapan arsitektur microservice pada proyek ini diharapkan dapat memberikan gambaran mengenai bagaimana konsep integrasi aplikasi enterprise dapat digunakan untuk membangun sistem yang modular, scalable, dan mampu menangani proses bisnis secara lebih efektif.

1.2 Tujuan Proyek

Tujuan dari pengembangan Warkop Management System adalah sebagai berikut:

1. Membangun sistem manajemen warkop berbasis arsitektur microservice yang terdiri atas beberapa layanan independen.

2. Mengimplementasikan konsep Enterprise Application Integration melalui komunikasi sinkron menggunakan REST API antar layanan.
3. Mengimplementasikan komunikasi asinkron berbasis event menggunakan RabbitMQ untuk meningkatkan efisiensi proses bisnis.
4. Mengimplementasikan GraphQL menggunakan Hasura sebagai mekanisme pengambilan data yang lebih fleksibel.
5. Menerapkan containerization menggunakan Docker untuk memudahkan deployment dan pengelolaan seluruh layanan beserta database yang digunakan.
6. Meningkatkan modularitas, skalabilitas, dan maintainability sistem melalui pemisahan tanggung jawab pada setiap layanan.

1.3 Deskripsi Proyek

Warkop Management System merupakan sistem yang dikembangkan untuk mengelola proses transaksi pada sebuah warung kopi dengan pendekatan arsitektur microservice. Sistem terdiri atas empat layanan utama, yaitu User Service untuk mengelola data pengguna, Product Service untuk mengelola data produk dan stok, Order Service sebagai pusat orkestrasi transaksi, serta Payment Service untuk menangani proses pembayaran.

Dalam implementasinya, Order Service berperan sebagai penghubung utama antar layanan. Validasi data pengguna dan produk dilakukan secara sinkron melalui REST API menggunakan HTTP client. Setelah pesanan berhasil dibuat, Order Service akan mengirimkan event ke RabbitMQ untuk memproses pengurangan stok produk dan pembayaran secara asinkron. Pendekatan ini memungkinkan proses transaksi berjalan lebih efisien dan mengurangi ketergantungan langsung antar layanan.

Selain itu, sistem menyediakan layanan GraphQL menggunakan Hasura yang terhubung dengan basis data PostgreSQL pada Order Service. Dengan GraphQL, pengguna dapat melakukan query dan mutasi data secara lebih fleksibel melalui satu endpoint tunggal.

Seluruh layanan dan basis data dijalankan menggunakan Docker sehingga proses deployment, pengelolaan, dan integrasi antar komponen dapat dilakukan secara lebih terstruktur. Dengan kombinasi teknologi tersebut, Warkop Management System menjadi implementasi nyata dari konsep Enterprise Application Integration yang memanfaatkan komunikasi sinkron, komunikasi asinkron berbasis event, GraphQL, serta containerization dalam lingkungan microservice.

BAB II

ARSITEKTUR SISTEM

2.1 Gambaran Arsitektur Sistem

Warkop Management System dibangun menggunakan arsitektur microservice, yaitu pendekatan yang membagi sistem menjadi beberapa layanan independen yang memiliki tanggung jawab spesifik. Setiap layanan dapat dikembangkan, dijalankan, dan dipelihara secara terpisah sehingga meningkatkan modularitas, skalabilitas, serta kemudahan dalam pengelolaan sistem.

Pada proyek ini, sistem terdiri atas empat layanan utama, yaitu User Service, Product Service, Order Service, dan Payment Service. Masing masing layanan memiliki basis data tersendiri sesuai dengan prinsip database per service. User Service, Product Service, dan Payment Service menggunakan MySQL, sedangkan Order Service menggunakan PostgreSQL.

Order Service berperan sebagai pusat orkestrasi transaksi. Ketika pengguna membuat pesanan, Order Service akan melakukan komunikasi sinkron dengan User Service dan Product Service melalui REST API untuk memvalidasi data pengguna dan produk secara langsung. Setelah proses validasi berhasil, data pesanan disimpan ke dalam basis data dan Order Service akan mengirimkan event ke RabbitMQ untuk menjalankan proses lanjutan secara asinkron.

Komunikasi asinkron berbasis event diterapkan menggunakan RabbitMQ sebagai message broker. Event yang dikirim oleh Order Service akan diproses oleh Product Service untuk memperbarui stok produk dan oleh Payment Service untuk memproses pembayaran. Pendekatan ini memungkinkan setiap layanan bekerja secara independen tanpa saling menunggu sehingga performa sistem menjadi lebih baik.

Selain komunikasi melalui REST API dan RabbitMQ, sistem juga menyediakan layanan GraphQL menggunakan Hasura yang terhubung dengan basis data PostgreSQL pada Order Service. GraphQL memungkinkan pengguna melakukan query dan mutasi data secara lebih fleksibel melalui satu endpoint, sehingga kebutuhan data dapat disesuaikan dengan permintaan pengguna. Seluruh layanan beserta basis data dan RabbitMQ dijalankan di dalam container Docker dan dikelola menggunakan Docker Compose. Containerization memungkinkan proses deployment, konfigurasi, serta

integrasi antar komponen dilakukan secara lebih mudah dan konsisten pada berbagai lingkungan.

Secara keseluruhan, arsitektur microservice yang diterapkan pada Warkop Management System mengintegrasikan komunikasi sinkron melalui REST API, komunikasi asinkron berbasis event menggunakan RabbitMQ, penyediaan data yang fleksibel melalui GraphQL, serta containerization menggunakan Docker. Arsitektur ini menghasilkan sistem yang lebih modular, scalable, dan mudah untuk dikembangkan maupun dipelihara.

2.2 Diagram Arsitektur Sistem

Menampilkan diagram yang berisi:

- Product Service
- User Service
- Order Service
- Payment Service

2.3 Teknologi yang Digunakan

Teknologi	Fungsi
Laravel	Framework Backend
MySQL, PostgreSQL	Database
RabbitMQ	Message Broker
Docker	Containerization
Postman	API Testing dan Dokumentasi
GitHub	Version Control
Hasura	Integrasi GraphQL

2.4 Komunikasi Antar Service

Pada Warkop Management System, komunikasi antar service dilakukan menggunakan dua mekanisme, yaitu komunikasi sinkron melalui REST API dan komunikasi asinkron melalui RabbitMQ Message Queue. Kombinasi kedua mekanisme tersebut memungkinkan sistem bekerja secara lebih modular dan efisien sesuai dengan konsep Enterprise Application Integration.

1. REST API

REST API digunakan sebagai mekanisme komunikasi sinkron antar service. Pada komunikasi sinkron, service pengirim akan menunggu respons dari service tujuan sebelum melanjutkan proses berikutnya. Mekanisme ini digunakan ketika diperlukan validasi data secara langsung atau real-time.

Dalam implementasi sistem, Order Service berperan sebagai orchestrator yang berkomunikasi dengan User Service dan Product Service menggunakan HTTP request. Ketika pengguna membuat pesanan, Order Service terlebih dahulu melakukan validasi terhadap data pengguna dengan mengirimkan permintaan ke User Service. Selanjutnya, Order Service melakukan validasi data produk dan ketersediaan stok melalui Product Service.

Komunikasi sinkron ini memastikan bahwa data yang digunakan dalam proses transaksi telah valid sebelum pesanan disimpan ke dalam basis data. Dengan demikian, integritas data antar service dapat terjaga dengan baik.

Alur komunikasi sinkron dapat dijelaskan sebagai berikut:

- Client mengirimkan permintaan pembuatan pesanan ke Order Service.
- orderService memanggil User Service untuk memvalidasi user_id.
- orderService memanggil Product Service untuk memvalidasi product_id dan stok produk.
- userService dan Product Service mengirimkan respons kepada Order Service.

2. RabbitMQ Message Broker

RabbitMQ digunakan sebagai media komunikasi asinkron antar service dengan menerapkan konsep event driven architecture. Pada komunikasi asinkron, service pengirim tidak perlu menunggu proses pada service penerima selesai sehingga kinerja sistem menjadi lebih efisien.

Setelah data pesanan berhasil disimpan, Order Service akan bertindak sebagai publisher yang mengirimkan event ke RabbitMQ melalui beberapa queue. Event tersebut kemudian diproses oleh service lain yang berperan sebagai consumer.

BAB III

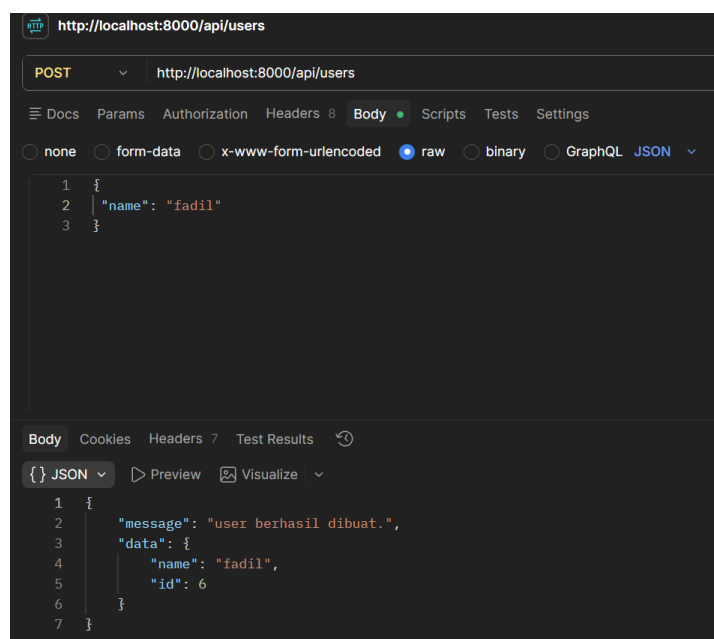
IMPLEMENTASI MICROSERVICE

3.1 User Service

User Service adalah layanan yang memiliki fungsi untuk mengelola seluruh informasi terkait user atau pelanggan yang melakukan transaksi pada warkop untuk melakukan pesanan. Pada User Service menyediakan beberapa layanan untuk mengelola user yang melakukan transaksi atau order pada Warkop mulai dari menambahkan user baru, menampilkan user secara keseluruhan atau berdasarkan ID user saja, layanan User Service juga menyediakan fitur mengupdate dan delete pada user. User Service menggunakan pendekatan berbasis RESTful API

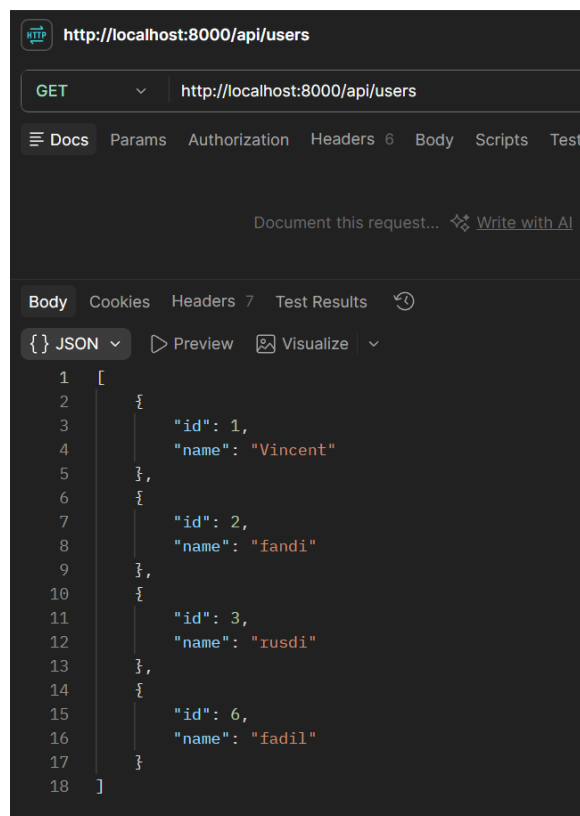
3.1.1 Create User (POST)

Fitur pada Create User memiliki tujuan untuk menambahkan nama baru user dalam database, pada layanan ini user memberikan informasi terkait nama yang akan digunakan user untuk melakukan order. Berdasarkan pengujian yang telah dilakukan di postman, user yang telah menginputkan informasi terkait nama yang akan digunakan dalam proses pengorderan akan tersimpan ke dalam database, setelah tersimpan ke dalam database sistem akan mengembalikan dan memberikan response bahwa user berhasil ditambahkan dan tambahan informasi nama dan juga ID user.



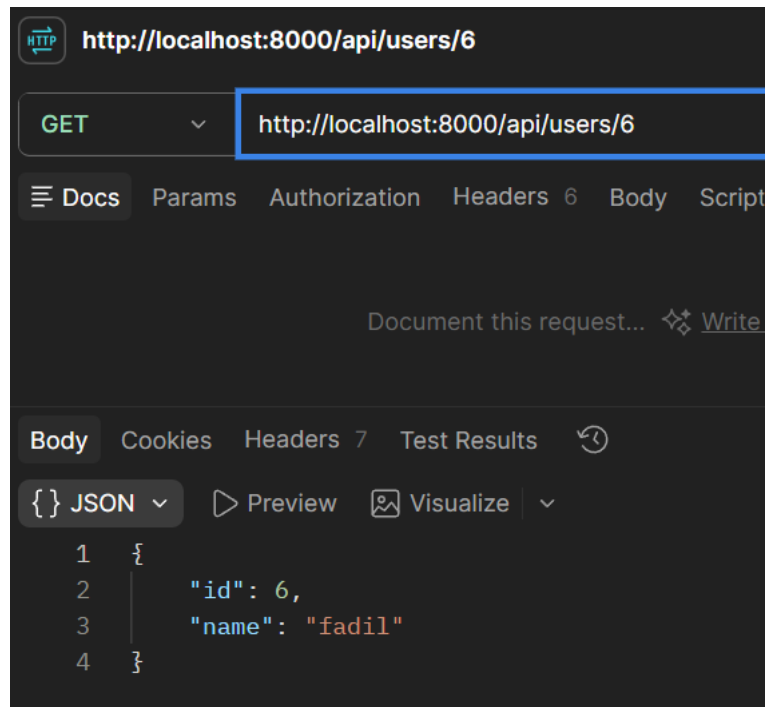
3.1.2 Get All User (GET)

Pada fitur get all user ini adalah proses untuk menarik atau memperoleh seluruh data user yang telah tercatat dan tersimpan pada UserService, ini bertujuan untuk mempermudah user dan juga service lain yang berkomunikasi untuk melihat keseluruhan data user tanpa harus mencari satu persatu menggunakan ID user. Berdasarkan pengujian yang telah dilakukan di Postman, menunjukkan bahwa sistem mampu menampilkan response JSON yang berisikan seluruh data nama dan ID user yang ada dalam database



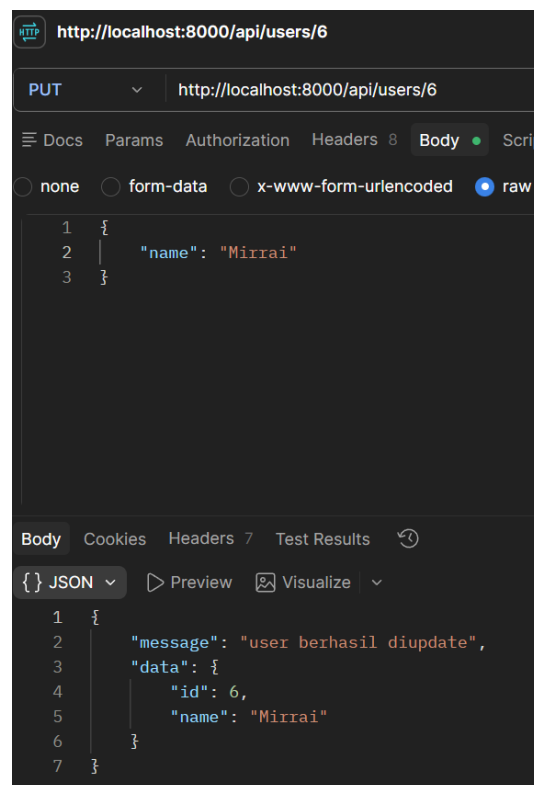
3.1.3 Get User by ID (GET)

Endpoint kali ini masih berfokus di memperoleh informasi nama dan id user yang telah terdaftar di database yang hendak melakukan order, perbedaannya kali ini sistem hanya memperoleh informasi user menggunakan ID yang sesuai untuk memanggil 1 user yang spesifik. Sesuai dengan yang telah diuji di Postman, sistem menunjukkan response JSON berupa data satu user sesuai id user yang ditambahkan pada URL.



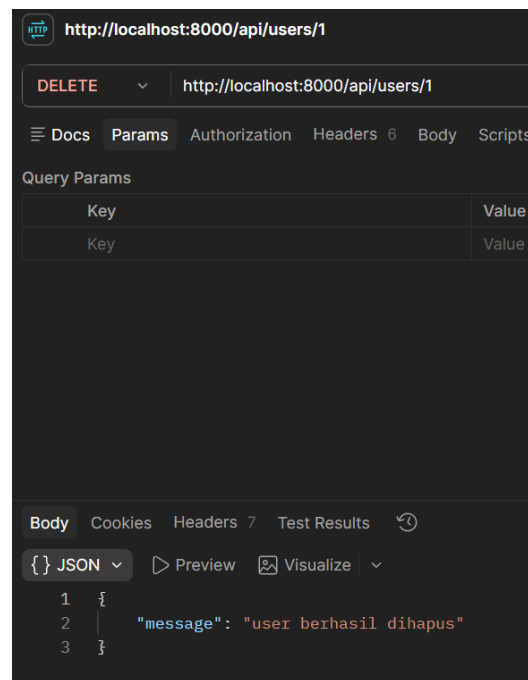
3.1.4 Update User (PUT)

Endpoint update user memiliki fungsi utama untuk melakukan perubahan pada data user yang sudah ada ke perubahan yang baru



3.1.5 Delete User (DELETE)

A

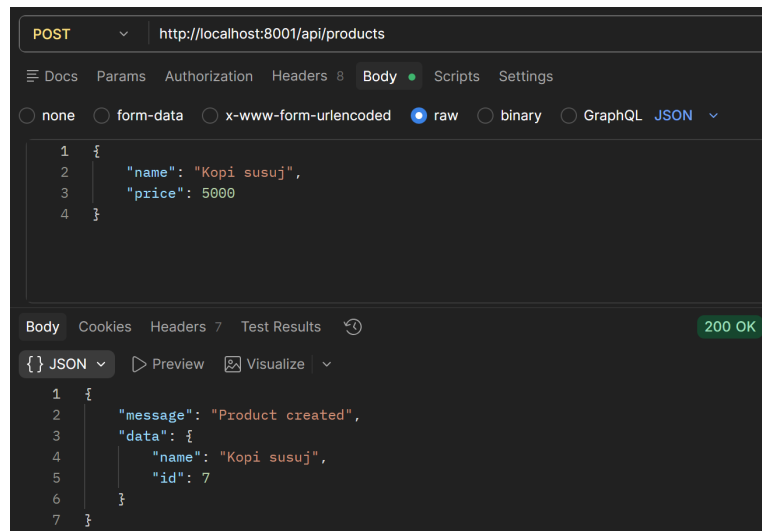


3.2 Product Service

Product Service merupakan layanan yang berfungsi untuk mengelola seluruh informasi produk yang tersedia pada Warkop. Service ini menyediakan berbagai operasi pengelolaan data produk, mulai dari penambahan produk baru, menampilkan daftar produk, melihat detail produk, memperbarui informasi produk, hingga menghapus produk dari sistem. Melalui pendekatan RESTful API, setiap fungsi pada Product Service dapat diakses menggunakan endpoint HTTP yang berbeda sesuai kebutuhan pengguna maupun service lain yang terintegrasi dalam sistem.

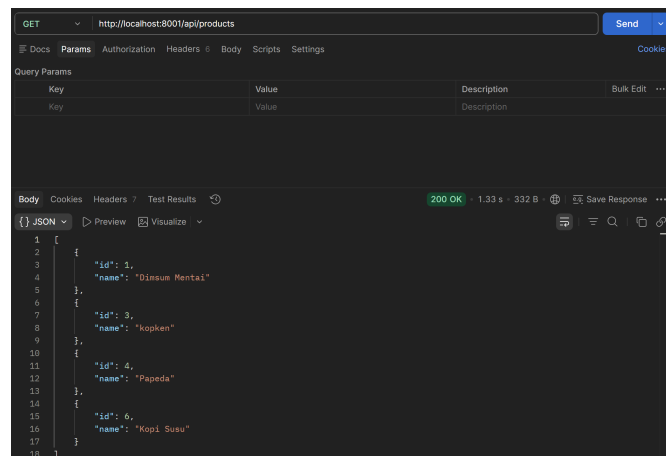
3.2.1 Create Product (POST)

Fitur Create Product digunakan untuk menambahkan data produk baru ke dalam database. Pada proses ini, pengguna mengirimkan informasi produk berupa nama dan harga produk melalui request API. Berdasarkan hasil pengujian menggunakan Postman, sistem berhasil menerima data yang dikirimkan dan menyimpannya ke dalam database. Setelah proses penyimpanan selesai, sistem mengembalikan response yang berisi informasi produk yang baru ditambahkan beserta ID yang dibuat secara otomatis oleh sistem.



3.2.2 Get All Product (GET)

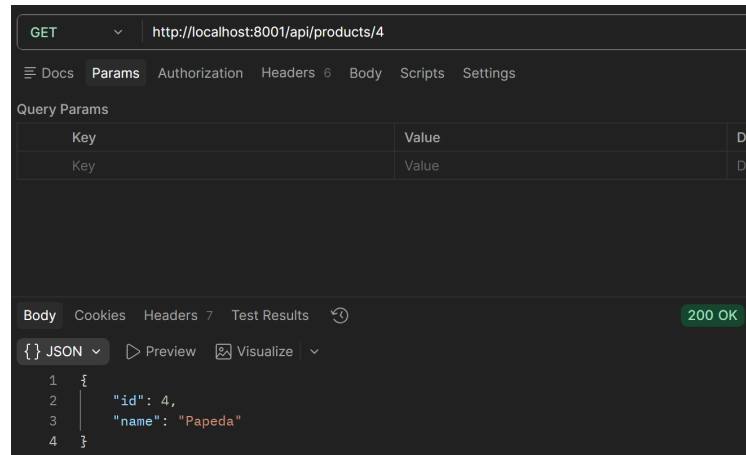
Endpoint Get All Products digunakan untuk memperoleh seluruh data produk yang tersimpan pada Product Service. Endpoint ini memudahkan pengguna maupun service lain untuk melihat daftar produk yang tersedia tanpa perlu memasukkan parameter tambahan. Hasil pengujian menunjukkan bahwa sistem mampu menampilkan seluruh data produk dalam format JSON. Informasi yang ditampilkan meliputi identitas produk, nama produk, dan harga produk yang tersimpan di database.



3.2.3 Get Product by ID (GET)

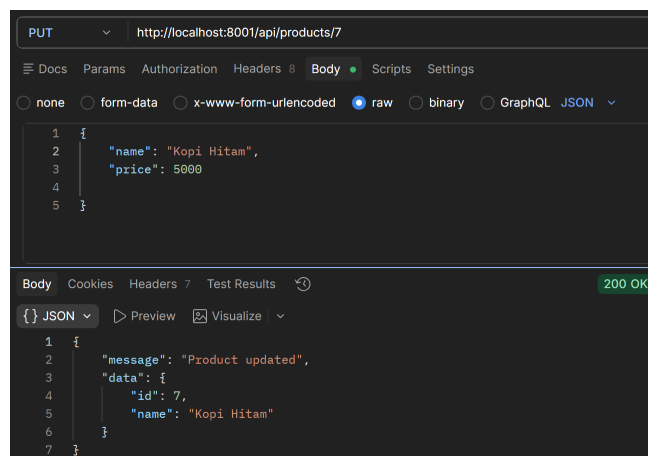
Fitur Get Product by ID digunakan untuk menampilkan informasi produk tertentu berdasarkan ID yang diberikan. Endpoint ini berguna ketika pengguna hanya membutuhkan detail dari satu produk secara spesifik. Dari hasil pengujian yang dilakukan, sistem berhasil menampilkan data produk

yang sesuai dengan ID yang diminta. Informasi yang diterima mencakup data lengkap produk yang tersimpan pada database.



3.2.4 Update Product (PUT)

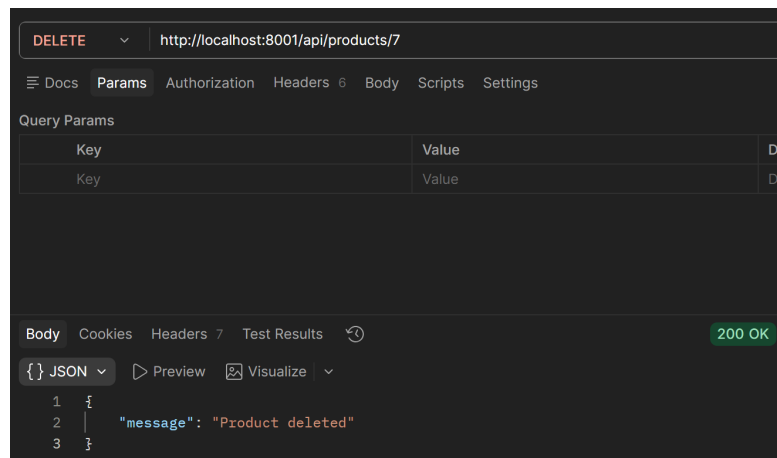
Endpoint Update Product digunakan untuk melakukan perubahan terhadap data produk yang sudah ada. Informasi yang dapat diperbarui meliputi nama produk maupun harga produk sesuai kebutuhan. Pengujian dilakukan dengan mengirimkan data perubahan melalui Postman. Sistem berhasil memproses perubahan tersebut dan mengembalikan response yang menunjukkan bahwa data produk telah berhasil diperbarui sesuai dengan nilai terbaru yang diberikan.



3.2.5 Delete Product (DEL)

Fitur Delete Product digunakan untuk menghapus data produk berdasarkan ID tertentu. Endpoint ini berfungsi menjaga data produk agar tetap relevan dengan kondisi yang ada pada sistem. Hasil pengujian

menunjukkan bahwa produk yang dipilih berhasil dihapus dari database. Setelah proses penghapusan selesai, sistem memberikan response sebagai konfirmasi bahwa data produk telah berhasil dihapus.



3.3 Order Service

A

3.3.1 A

A

3.3.2

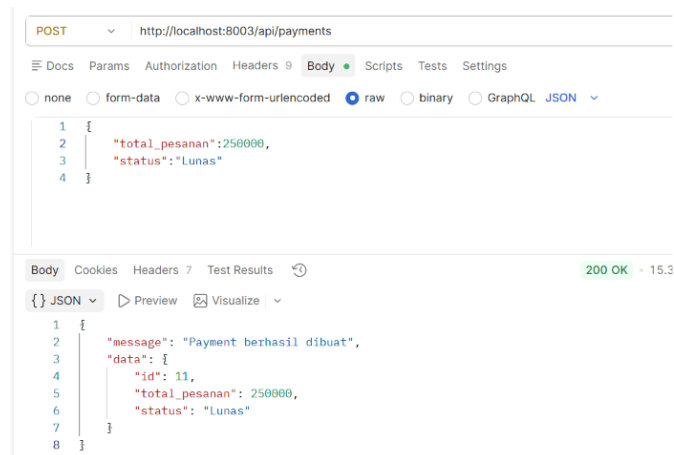
3.4 Payment Service

Payment Service merupakan salah satu layanan dalam sistem Warkop Management System yang bertanggung jawab untuk mengelola proses pembayaran. Service ini berfungsi untuk mencatat, memperbarui, menampilkan, dan menghapus data pembayaran yang terjadi dalam sistem.

Payment Service dibangun dengan konsep RESTful API sehingga setiap fitur diakses melalui endpoint HTTP. Setiap endpoint memiliki fungsi spesifik untuk mendukung proses pengelolaan data pembayaran secara terstruktur.

3.4.1 Create Payment (Post)

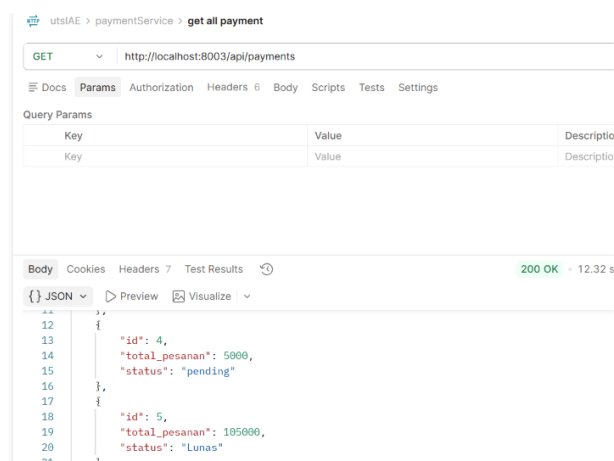
Create Payment merupakan endpoint yang digunakan untuk membuat data pembayaran baru dalam sistem Payment Service. Endpoint ini berfungsi untuk mencatat setiap transaksi pembayaran yang dilakukan oleh pengguna, termasuk informasi seperti total pesanan dan status pembayaran yang diberikan.



Hasil pengujian pada Postman menunjukkan bahwa data pembayaran berhasil dibuat dan tersimpan di dalam sistem. Data yang dikirimkan berupa `total_pesanan` dan `status` diproses oleh sistem dan dikembalikan dalam response API. Nilai `total_pesanan` sebesar 250000 berhasil tersimpan dalam data transaksi pembayaran dengan status “Lunas”, serta sistem memberikan ID otomatis untuk setiap data yang dibuat.

3.4.2 Get All Payments (Get)

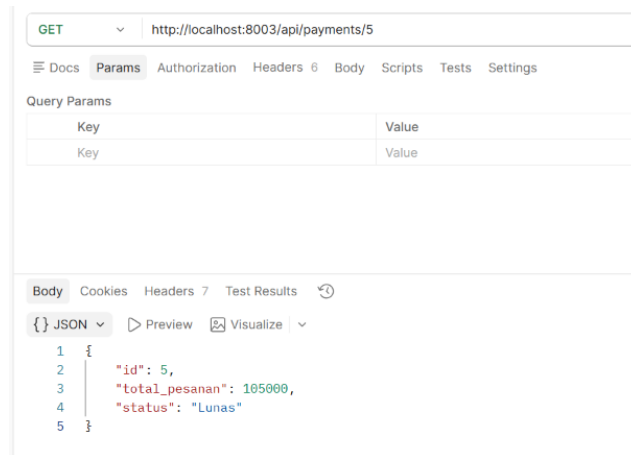
Get All Payment merupakan endpoint yang digunakan untuk menampilkan seluruh data pembayaran yang tersimpan di dalam sistem Payment Service. Endpoint ini berfungsi untuk mengambil semua transaksi pembayaran tanpa memerlukan parameter tertentu.



Hasil pengujian pada Postman menunjukkan bahwa sistem berhasil mengembalikan seluruh data payment dalam bentuk JSON array. Setiap data pembayaran berisi informasi seperti `id`, `total_pesanan`, dan `status` pembayaran seperti “Lunas” atau “Pending” yang tersimpan di database.

3.4.3 Get Payment by ID (Get)

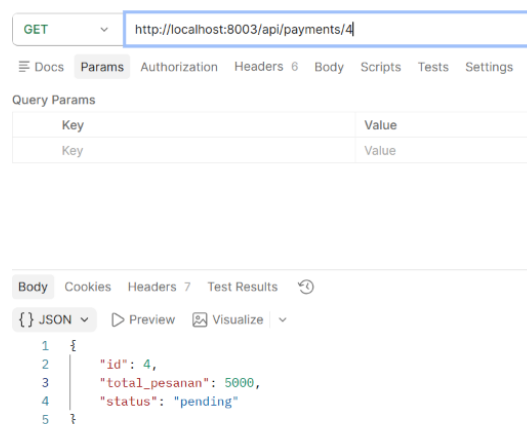
Get Payment by ID merupakan endpoint yang digunakan untuk mengambil data pembayaran berdasarkan ID tertentu. Endpoint ini berfungsi untuk menampilkan detail satu transaksi pembayaran secara spesifik sesuai dengan parameter ID yang dikirimkan melalui URL.



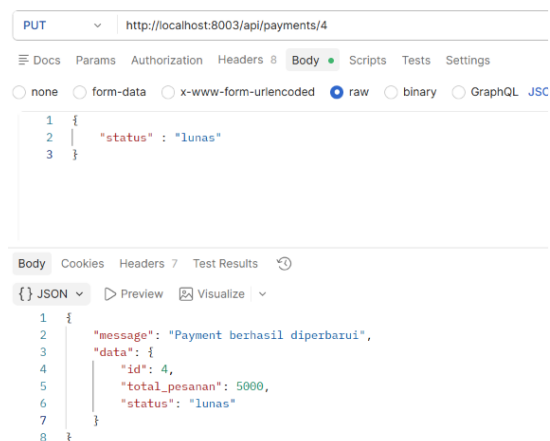
Hasil pengujian pada Postman menunjukkan bahwa sistem berhasil mengembalikan data pembayaran sesuai dengan ID yang diminta. Data yang ditampilkan mencakup informasi seperti id transaksi, total pesanan, dan status pembayaran

3.4.4 Update Payment (Put)

Update Payment merupakan endpoint yang digunakan untuk memperbarui data pembayaran yang sudah tersimpan di dalam sistem berdasarkan ID tertentu. Endpoint ini memungkinkan perubahan informasi pembayaran, seperti status pembayaran, sesuai dengan kebutuhan transaksi.



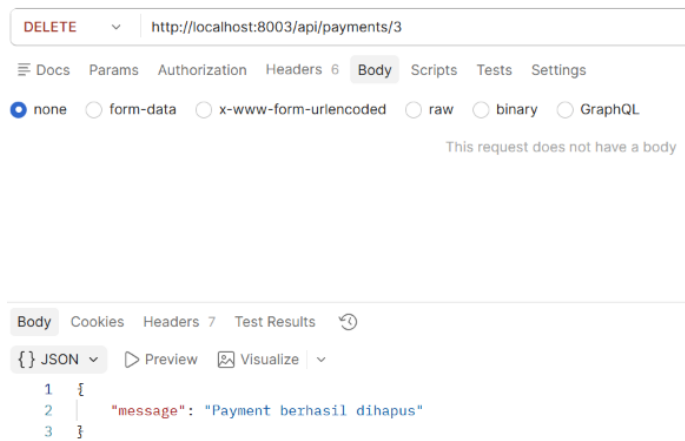
Hasil pengujian endpoint Update Payment dilakukan menggunakan Postman dengan mengirimkan request ke endpoint PUT /api/payments/{id}. Pada pengujian awal, data pembayaran yang dipilih memiliki status "pending" dengan total pesanan sebesar 5000. Setelah request update dikirim dengan perubahan status menjadi "lunas", sistem berhasil memproses perubahan tersebut.



Hal ini dibuktikan dengan respons dari sistem yang menampilkan pesan “Payment berhasil diperbarui” serta data pembayaran yang telah diperbarui, dimana status transaksi berubah dari pending menjadi lunas tanpa mengubah data utama lainnya seperti ID dan total pesanan.

3.4.5 Delete Payment (Delete)

Pengujian endpoint Delete Payment dilakukan untuk memastikan bahwa sistem dapat menghapus data pembayaran yang sudah tidak diperlukan berdasarkan ID tertentu. Proses ini bertujuan untuk menguji apakah data yang dipilih benar-benar dapat dihapus dari database melalui request yang dikirimkan menggunakan Postman.



Setelah request dikirimkan, sistem berhasil memproses penghapusan data tersebut. Hal ini dibuktikan dengan munculnya respons dari sistem berupa pesan **“Payment berhasil dihapus”**, yang menandakan bahwa data pembayaran dengan ID yang dipilih telah berhasil dihapus dari database dan tidak lagi tersimpan di dalam sistem.

BAB IV IMPLEMENTASI DAN PENGUJIAN SISTEM

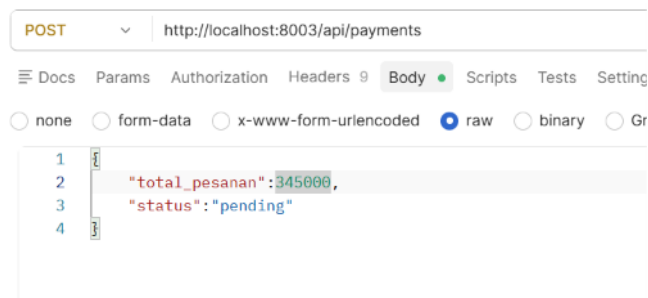
4.1 Implementasi RabbitMQ

Implementasi RabbitMQ ditunjukkan melalui RabbitMQ Management Dashboard yang menampilkan queue yang aktif dalam sistem. Pada sistem ini terdapat queue `payment_queue` yang digunakan untuk menampung pesan dari Payment Service sebelum diproses oleh Payment Worker.

4.4.1 Postman – CREATE PAYMENT (Trigger awal)

Pada tahap ini dilakukan pengujian awal untuk membuat data pembayaran menggunakan Postman. Client mengirimkan request dengan metode POST ke endpoint `/api/payments` dengan data berupa `total_pesanan` dan status pembayaran. Data ini merepresentasikan transaksi pembayaran yang dilakukan oleh pengguna dalam sistem.

Setelah request dikirimkan, Payment Service akan menerima data tersebut dan memprosesnya untuk kemudian disimpan ke dalam sistem. Selain itu, Payment Service juga akan mengirimkan event ke RabbitMQ untuk diproses secara asinkron oleh Payment Worker melalui queue yang telah disediakan.



4.4.2 Response Postman (hasil berhasil)

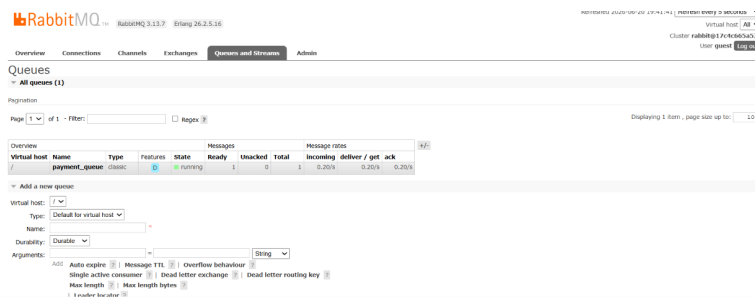
Gambar menunjukkan hasil response dari Postman setelah pengiriman request pembuatan data pembayaran. Sistem mengembalikan pesan “Payment berhasil dibuat” beserta data transaksi yang mencakup `id`, `total_pesanan`, dan status pembayaran yang masih bernilai “pending”, yang menandakan bahwa proses penyimpanan data berhasil dilakukan oleh Payment Service.

```
Body Cookies Headers 7 Test Results 200 OK
{ } JSON Preview Visualize
1 {
2   "message": "Payment berhasil dibuat",
3   "data": {
4     "id": 17,
5     "total_pesanan": 345000,
6     "status": "pending"
7   }
8 }
```

4.4.3 RabbitMQ Dashboard (QUEUE MUNCUL / READY > 0)

Pada tahap ini, sistem RabbitMQ sudah berhasil menerima pesan dari Payment Service melalui queue *payment_queue*. Pesan tersebut dikirim setelah proses pembuatan pembayaran dilakukan melalui endpoint Create Payment. Data transaksi yang dikirim oleh service kemudian ditampung terlebih dahulu di dalam queue sebelum diproses oleh Payment Worker.

Kondisi ini menunjukkan bahwa mekanisme komunikasi asinkron pada sistem sudah berjalan dengan baik. Adanya data pada queue (READY > 0) menandakan bahwa pesan dari Payment Service berhasil masuk ke RabbitMQ dan sedang menunggu untuk diproses oleh consumer. Hal ini membuktikan bahwa integrasi antara Payment Service dan RabbitMQ telah berhasil diimplementasikan dalam arsitektur microservice yang digunakan.



4.4.4 Docker Terminal (Payment Worker Running)

Pada tahap ini, Payment Worker telah berhasil dijalankan di dalam container Docker dan dalam keadaan aktif (running), yang menandakan bahwa service consumer sudah siap untuk memproses pesan dari RabbitMQ. Worker ini berfungsi untuk mengambil data pembayaran yang masuk ke queue *payment_queue* kemudian memprosesnya secara otomatis sesuai dengan alur sistem, sehingga seluruh proses berjalan secara asinkron tanpa menghambat service utama.

☐	cafe-docker-compose	-	-	-	4			
☐	payment-service	4c170156e0fc	cafe-docke	8003:8003	0.			
☐	payment-worker	92ed14c01b24	cafe-docke					

4.4.5 RabbitMQ Dashboard setelah diproses (READY = 0)

Pada tahap ini, hasil implementasi RabbitMQ menunjukkan bahwa seluruh pesan yang sebelumnya masuk ke dalam queue *payment_queue* telah berhasil diproses oleh Payment Worker. Hal ini ditandai dengan nilai READY yang bernilai 0, yang berarti tidak ada lagi pesan yang menunggu dalam antrian.

Kondisi tersebut menunjukkan bahwa mekanisme komunikasi asinkron antara Payment Service, RabbitMQ, dan Payment Worker telah berjalan dengan baik, di mana setiap data pembayaran yang dikirim sebelumnya sudah berhasil diambil dan diproses hingga selesai tanpa ada penumpukan pesan di dalam queue.

RabbitMQ 3.13.7 Erlang 26.2.5.16

Refreshed 2026-06-20 19:42:16 Refresh every 5 seconds

Virtual host: All

Cluster rabbit@17-c4c865a572

User guest Log out

Overview Connections Channels Exchanges **Queues and Streams** Admin

Queues

All queues (1)

Page 1 of 1 - Filter: ☐ Regex

Displaying 1 item, page size up to: 100

Virtual host	Name	Type	Features	State	Ready	Unacked	Total	Message rate
/	payment_queue	classic		running	0	0	0	0.00/s 0.00/s 0.00/s

Add a new queue

Virtual host: /

Type: Default for virtual host

Name:

Durability:

Arguments:

Auto expire ☐ Message TTL ☐ Overflow behaviour ☐

Single active consumer ☐ Dead letter exchange ☐ Dead letter routing key ☐

Max length ☐ Max length bytes ☐

Leader locator ☐

4.2 Implementasi Hasura GraphQL

A

BAB V PENUTUP

5.1 Kesimpulan

5.2 Saran

LAMPIRAN